# DevOps & Cloud

Interview Preparation Guide

Top 20 Real-World Interview Questions

---

**Sanket Ajay Chopade**

Junior DevOps Engineer | EKS · Terraform · GitHub Actions · ArgoCD · Helm

Hisan Labs Private Limited — Internship (Sep 2025 – May 2026)

| AWS EKS | Fargate | ECR | Terraform | GitHub Actions | ArgoCD | Helm | Prometheus | Grafana | Trivy |
|---|---|---|---|---|---|---|---|---|---|

Question 1 of 20

## Tell me about a production incident you handled and how you resolved it.

**Why They Ask:** Tests crisis communication, ownership, and structured thinking under pressure.

### ■ Situation

- During my internship at Hisan Labs, our EKS Fargate cluster hosting the containerized 2048 game platform showed pod CrashLoopBackOff events during a deployment.
- The ECR image had a misconfigured entrypoint caused by an updated base image that changed the default shell path.

### ■ Actions Taken

- Ran kubectl describe pod and kubectl logs to identify the root cause immediately.
- Rolled back to the previous stable image tag using kubectl rollout undo deployment.
- Pinned the base image SHA in the Dockerfile to prevent silent upstream changes.
- Added a post-build smoke test in the GitHub Actions pipeline to validate container startup before push to ECR.

### ■ Outcome

- Service restored within ~15 minutes. Zero recurrence after the fix.
- Documented the incident in a post-mortem and introduced image pinning as a team standard.

---

★ *Tip: Use the STAR format. Interviewers want ownership + learning, not just technical steps.*

Question 2 of 20

## How would you design a CI/CD pipeline for a microservices application?

**Why They Ask:** Tests pipeline architecture thinking, tool knowledge, and understanding of service isolation.

### ■ Pipeline Stages

- Source: Developer pushes to feature branch -> PR triggers pipeline.
- Build: Docker image built per microservice; tagged with Git SHA.
- Test: Unit tests, integration tests, SonarQube SAST scan, Trivy image vulnerability scan.
- Push: On merge to main, image pushed to ECR/ACR with version tag.
- Deploy (Dev/Staging): ArgoCD or Helm chart update triggers GitOps sync to cluster.
- Deploy (Prod): Manual approval gate or automated canary via Argo Rollouts.
- Observe: Prometheus + Grafana dashboards validate SLOs post-deploy.

### ■ Tools Used in Internship

- GitHub Actions for CI, ArgoCD for GitOps CD, Helm for Kubernetes packaging.
- Trivy for container scanning, SonarQube for code quality, ECR for image registry.

### ■ Key Principles

- Each microservice has its own pipeline — no monolithic pipeline.
- Artifact immutability: same image tag promoted across environments.
- Rollback is a first-class step, not an afterthought.

*★ Tip: Draw the pipeline mentally while answering. Mention security scanning early — it signals maturity.*

Question 3 of 20

## What is the difference between Docker and Kubernetes?

**Why They Ask:** Fundamental concept check — don't confuse 'containerization' with 'orchestration'.

### ■ Docker

- A container runtime and image build tool.
- Packages an application and its dependencies into a portable image.
- Runs containers on a single host; no built-in high availability or scaling.
- Core components: Dockerfile, docker build, docker run, Docker Hub/ECR.

### ■ Kubernetes

- A container orchestration platform that manages containers at scale across multiple nodes.
- Provides: auto-scaling (HPA), self-healing (restarts failed pods), service discovery, rolling updates, secrets management.
- Core objects: Pod, Deployment, Service, ConfigMap, Namespace, Ingress.
- Kubernetes does not build images — it pulls and runs them (from ECR, Docker Hub, etc.).

### ■ Analogy

- Docker = shipping container. Kubernetes = the port that manages thousands of containers, routes them, and replaces damaged ones automatically.

---

*★ Tip: Mention that in your internship you used both — Docker to build and ECR to store, Kubernetes (EKS Fargate) to orchestrate.*

Question 4 of 20

## How do you manage secrets and sensitive information in your deployments?

**Why They Ask:** Critical security question — wrong answers (hardcoding, plain env vars) are instant red flags.

### ■ What NOT to do

- Never hardcode secrets in Dockerfiles, source code, or Helm values files.
- Never store secrets in plain-text ConfigMaps in Kubernetes.

### ■ Correct Approaches

- AWS Secrets Manager / Parameter Store: Store secrets centrally; applications fetch at runtime.
- IRSA (IAM Roles for Service Accounts): Grant EKS pods permissions to access AWS Secrets Manager without static credentials.
- Kubernetes native Secrets: Base64-encoded but not encrypted at rest by default — combine with envelope encryption using AWS KMS.
- External Secrets Operator: Syncs AWS Secrets Manager values into Kubernetes Secrets automatically.
- Vault (HashiCorp): Enterprise-grade secrets engine with dynamic credentials, audit logs, and lease management.

### ■ In My Internship

- Used IRSA to authenticate EKS Fargate pods to AWS services without embedding access keys.
- Identified External Secrets Operator as a gap — actively learning it now.

---

★ *Tip: Honesty about gaps (ESO, Vault) + showing you know IRSA is strong. Don't fake Vault expertise.*

Question 5 of 20

## Explain the difference between Infrastructure as Code and Configuration Management.

---

**Why They Ask:** Tests whether you understand the boundary between provisioning and post-provisioning.

### ■ Infrastructure as Code (IaC)

- Defines and provisions cloud infrastructure: VPCs, EC2 instances, EKS clusters, S3 buckets.
- Tools: Terraform, AWS CloudFormation, Pulumi.
- Declarative: you describe the desired end state; the tool figures out how to get there.
- Terraform used in internship to provision EKS cluster, VPC, ECR, IAM roles via .tf files.

### ■ Configuration Management

- Manages software state on already-provisioned servers: installs packages, applies configs, manages services.
- Tools: Ansible, Chef, Puppet.
- Ansible used in internship for post-provisioning tasks: installing dependencies, applying OS-level config.

### ■ Key Distinction

- IaC = 'create a server'. Configuration management = 'configure that server'.
- In modern cloud/container workflows, configuration management is less critical — containers bake config into images.

---

★ *Tip: Mention Terraform + Ansible together — shows you know the complementary workflow.*

Question 6 of 20

## How would you perform a zero-downtime deployment?

**Why They Ask:** Tests deployment strategy knowledge and Kubernetes-specific implementation.

### ■ Kubernetes Rolling Update (Default)

- Set strategy: RollingUpdate with maxUnavailable: 0 and maxSurge: 1 in Deployment spec.
- Kubernetes gradually replaces old pods with new ones — traffic never drops to zero.
- Health checks (readinessProbe) gate traffic: new pods receive traffic only when healthy.

### ■ Blue-Green Deployment

- Run two identical environments (Blue = live, Green = new version).
- Switch traffic at the load balancer/Ingress level after Green passes all tests.
- Instant rollback: just switch traffic back to Blue.

### ■ Canary Deployment

- Route a small % of traffic (e.g., 5%) to the new version.
- Monitor error rate, latency, and business metrics.
- Gradually increase traffic if stable; rollback if anomalies detected.

### ■ Supporting Practices

- readinessProbe and livenessProbe configured on every pod.
- PodDisruptionBudgets to prevent too many pods going down simultaneously.
- Feature flags to decouple deployment from feature release.

★ *Tip: Mention readinessProbe explicitly — many freshers miss this and it's the actual mechanism.*

Question 7 of 20

## What steps would you take if a Kubernetes pod keeps restarting?

**Why They Ask:** Practical troubleshooting question — expects a methodical diagnostic flow.

### ■ Step 1 – Describe the Pod

- kubectl describe pod -n
- Look at: Exit Code, Last State, Events section for OOMKilled, CrashLoopBackOff, ImagePullBackOff.

### ■ Step 2 – Check Logs

- kubectl logs --previous (gets logs from the crashed container)
- Look for: application errors, missing config, connection failures.

### ■ Step 3 – Common Root Causes & Fixes

- OOMKilled: Pod exceeded memory limit -> increase resources.limits.memory or fix memory leak.
- CrashLoopBackOff: App crashes on start -> check entrypoint, env vars, config mounts.
- ImagePullBackOff: Image not found or wrong tag -> verify ECR URI, IAM permissions, image tag.
- Missing ConfigMap/Secret: Volume mount fails -> kubectl get configmap / secret to verify existence.
- Failing livenessProbe: Probe too aggressive -> tune initialDelaySeconds and periodSeconds.

### ■ Step 4 – Exec into Container if Running

- kubectl exec -it -- /bin/sh
- Test connectivity, check env vars, validate config files inside the container.

---

★ *Tip: Always say 'kubectl describe first, then logs' — this sequence shows real troubleshooting discipline.*

Question 8 of 20

## How do you troubleshoot a failed deployment in production?

**Why They Ask:** Broader than pod restarts — tests end-to-end deployment debugging and rollback confidence.

### ■ Immediate Response

- Assess blast radius: is the service fully down or partially degraded?
- Do NOT make random changes — diagnose first.

### ■ Diagnostic Steps

- Check deployment status: kubectl rollout status deployment/
- Check ReplicaSet events: kubectl describe rs
- Check pod logs: kubectl logs --previous
- Check Ingress and Service: ensure correct selector labels and port mappings.
- Check recent changes in Git: what changed in the last commit?

### ■ Rollback

- kubectl rollout undo deployment/ to immediately revert to previous revision.
- In GitOps (ArgoCD): revert the Helm values commit and sync.

### ■ Post-Recovery

- Write a post-mortem: timeline, root cause, contributing factors, action items.
- Add pipeline test to catch the failure class before it reaches production.

*★ Tip: Interviewers want to hear 'rollback first, investigate second' — don't try to fix-forward in production blindly.*

Question 9 of 20

## Explain the difference between Blue-Green and Canary deployments.

**Why They Ask:** Conceptual + trade-off question — show you understand when to use each.

### ■ Blue-Green

- Two full environments: Blue (current live) and Green (new version).
- Traffic switches 100% from Blue to Green at once via load balancer.
- Pros: Instant rollback, clean separation, easy to test Green in isolation.
- Cons: Requires 2x infrastructure cost, all-or-nothing traffic switch.

### ■ Canary

- New version receives a small slice of real production traffic (e.g., 5-10%).
- Traffic percentage increases gradually based on metrics (error rate, p99 latency).
- Pros: Real user validation, gradual risk exposure, data-driven promotion.
- Cons: Complexity in traffic splitting; both versions run simultaneously (DB compatibility needed).

### ■ When to Use Which

- Blue-Green: When you need clean environment isolation and fast rollback (e.g., major version changes).
- Canary: When validating a change with real traffic at low risk (e.g., algorithm changes, A/B testing).

---

*★ Tip: Mention Argo Rollouts if asked for tooling — it implements both strategies natively on Kubernetes.*

Question 10 of 20

## How would you design a highly available architecture on AWS?

**Why They Ask:** Architecture design question — tests cloud fundamentals and redundancy thinking.

### ■ Core Principles

- Eliminate single points of failure (SPOF) at every layer.
- Design for failure: assume any component can fail at any time.

### ■ Compute Layer

- EKS with worker nodes spread across 3 Availability Zones (AZs).
- Auto Scaling Groups to replace failed nodes automatically.
- PodAntiAffinity rules to spread pods across AZs.

### ■ Networking Layer

- Application Load Balancer (ALB) across multiple AZs.
- Route 53 with health checks for DNS failover.
- VPC with public/private subnets; NAT Gateway per AZ.

### ■ Data Layer

- RDS Multi-AZ with automated failover.
- ElastiCache (Redis) in cluster mode across AZs.
- S3 for static assets — 11 nines durability, globally replicated.

### ■ Observability

- CloudWatch alarms + Prometheus/Grafana for real-time visibility.
- PagerDuty/SNS for alerting on SLO breaches.

---

*★ Tip: Always say 'multi-AZ' not 'multi-region' unless asked — multi-region is a different (and harder) problem.*

Question 11 of 20

## Describe a situation where you automated a repetitive manual task.

**Why They Ask:** Behavioral question — tests automation instinct and real impact delivery.

### ■ Situation

- At Hisan Labs, deploying the containerized 2048 application required manually building the Docker image, pushing to ECR, and updating the Kubernetes manifest every time — error-prone and slow.

### ■ Action

- Built a GitHub Actions CI/CD pipeline that triggered automatically on every push to main.
- Pipeline: checkout code -> docker build -> trivy scan -> push to ECR -> update Helm values -> ArgoCD sync.
- Added Slack notification step to alert the team on success or failure.

### ■ Result

- Deployment time dropped from ~20 minutes manual effort to ~4 minutes fully automated.
- Zero manual ECR push errors post-automation.
- Team adopted the same pipeline template for the Go Cloud-Native GitOps project.

*★ Tip: Quantify the time saved. Even rough numbers ('from 20 mins to 4 mins') land much better than vague statements.*

Question 12 of 20

## How do you monitor applications and infrastructure in production?

**Why They Ask:** Tests observability maturity — the 'three pillars' answer wins.

### ■ Three Pillars of Observability

- Metrics: Prometheus scrapes application and infrastructure metrics. Grafana dashboards visualize CPU, memory, error rate, request latency, pod count.
- Logs: Centralized log aggregation via CloudWatch Logs or ELK stack. Structured JSON logging in applications.
- Traces: Distributed tracing with Jaeger or AWS X-Ray to track requests across microservices.

### ■ Alerting

- Define SLOs (e.g., 99.9% availability, p99 latency < 500ms).
- Prometheus Alertmanager fires alerts when SLO error budget is burning too fast.
- PagerDuty or Opsgenie for on-call escalation.

### ■ Used in Internship

- Prometheus + Grafana deployed on EKS to monitor the containerized 2048 platform.
- Trivy for ongoing container vulnerability monitoring.

---

★ *Tip: Mention SLOs and error budgets if the role is SRE-leaning — it signals beyond-basics thinking.*

Question 13 of 20

## What would you do if your CI/CD pipeline suddenly started failing?

**Why They Ask:** Tests systematic debugging and ownership under pressure.

### ■ Step 1 – Identify the Failure Layer

- Check which stage failed: Source, Build, Test, Push, or Deploy.
- Read the full error log — don't skim.

### ■ Step 2 – Common Causes by Stage

- Source: Merge conflict, branch protection rule, webhook misconfiguration.
- Build: Changed base image, missing dependency, syntax error in Dockerfile.
- Test: New test failure, environment variable missing in CI context.
- Push: ECR authentication expired, IAM permission change, quota exceeded.
- Deploy: Helm chart error, Kubernetes API version mismatch, ArgoCD sync failure.

### ■ Step 3 – Check Recent Changes

- git log to see what changed in the last commit.
- Check if any infrastructure changes were made (Terraform apply, IAM updates).

### ■ Step 4 – Temporary Mitigation

- If a critical fix needs to reach production urgently, consider a manual deployment with documented approval.
- Do not bypass pipeline security gates (Trivy, SAST) in the process.

---

*★ Tip: Interviewers want methodical thinking, not panic. Say 'I look at the failed stage first, not the whole pipeline.'*

Question 14 of 20

## Explain the complete request flow from a user accessing an application to receiving a response.

**Why They Ask:** Full-stack networking question — tests depth across DNS, LB, app, and DB layers.

### ■ Step-by-Step Flow

- 1. User types URL -> Browser queries DNS resolver -> Route 53 returns ALB IP address.
- 2. Browser sends HTTPS request to ALB; TLS termination happens at ALB (ACM certificate).
- 3. ALB inspects Host header and path -> routes to target group based on Listener rules.
- 4. Request hits Kubernetes Ingress (NGINX or ALB Ingress Controller) -> routed to Service.
- 5. Service (ClusterIP) load-balances to one of the healthy Pods via kube-proxy/iptables.
- 6. Pod processes request: reads from cache (ElastiCache Redis) if available, else queries RDS.
- 7. Response travels back: Pod -> Service -> Ingress -> ALB -> TLS -> User browser.

### ■ Key Concepts to Name

- DNS resolution, TLS termination, Ingress controller, ClusterIP Service, kube-proxy, readinessProbe (ensures only healthy pods receive traffic).

---

*★ Tip: Draw this as a diagram in your head. Being able to walk through each hop shows real infrastructure understanding.*

Question 15 of 20

## How do you secure containerized applications?

**Why They Ask:** Security depth question — tests defense-in-depth thinking.

### ■ Image Security

- Use minimal base images (distroless, Alpine) to reduce attack surface.
- Scan images with Trivy before push to registry — block critical CVEs.
- Pin base image SHAs; never use :latest in production.

### ■ Runtime Security

- Run containers as non-root user (USER directive in Dockerfile).
- Set readOnlyRootFilesystem: true in SecurityContext.
- Drop all Linux capabilities; add only what's needed (e.g., NET_BIND_SERVICE).
- Apply PodSecurityAdmission policies (restricted profile).

### ■ Network Security

- Kubernetes NetworkPolicies: restrict pod-to-pod communication to only what's required.
- Use private subnets for pods; only expose via ALB Ingress.

### ■ Secrets & IAM

- IRSA for pod-level AWS permissions — no static access keys.
- Secrets injected via environment variables or volume mounts from AWS Secrets Manager.

### ■ Supply Chain Security

- Sign images with Cosign (Sigstore). Verify signatures at admission time with policy engine.

---

★ *Tip: Say 'defense in depth' — securing at build time, runtime, and network layer. It's the right framing.*

Question 16 of 20

## What are Terraform state files, and how do you manage them in a team environment?

**Why They Ask:** IaC maturity question — state mismanagement causes real production disasters.

### ■ What is State?

- Terraform state (terraform.tfstate) is a JSON file mapping your .tf configuration to real cloud resources.
- Terraform uses it to know what exists, what needs to change, and what to destroy.
- Without state, Terraform cannot determine drift or plan diffs.

### ■ Problems with Local State in Teams

- Two engineers running terraform apply simultaneously = state corruption.
- Local state is not shared — team members see inconsistent resource views.
- Accidental deletion of local state = Terraform loses track of all resources.

### ■ Best Practice: Remote State

- Store state in S3 bucket with versioning enabled (point-in-time recovery).
- Enable DynamoDB table for state locking — prevents concurrent applies.
- Encrypt state at rest with SSE-S3 or KMS (state contains sensitive resource IDs).

### ■ Terraform Backend Config

- backend "s3" { bucket = "tf-state-prod" key = "eks/terraform.tfstate" region = "ap-south-1" dynamodb_table = "tf-lock-table" encrypt = true }

*★ Tip: Mentioning DynamoDB locking specifically shows you understand concurrent team use, not just storage.*

Question 17 of 20

## How would you handle a sudden spike in application traffic?

**Why They Ask:** Tests scalability architecture and real-time response thinking.

### ■ Immediate / Auto Response (should be pre-configured)

- HPA (Horizontal Pod Autoscaler): Scales pods based on CPU/memory or custom metrics (RPS, queue depth).
- Cluster Autoscaler / Karpenter: Adds EC2 nodes when pods can't be scheduled.
- ALB scales automatically — no action needed.

### ■ Short-Term Manual Actions

- kubectl scale deployment --replicas= for immediate manual scale.
- Increase RDS read replicas if DB is the bottleneck.
- Enable ElastiCache caching layer to absorb read traffic.

### ■ Longer-Term Architecture

- CDN (CloudFront) for static assets — reduces origin load significantly.
- Rate limiting at Ingress layer (NGINX annotations or WAF rules).
- Async processing: offload heavy tasks to SQS queues + Lambda/worker pods.
- Load test regularly with k6 or Locust — don't discover limits under real traffic.

---

*★ Tip: Separate 'what should happen automatically' from 'what you do manually' — HPA is proactive, manual scale is a backup.*

Question 18 of 20

## What strategies do you use for backup and disaster recovery?

**Why They Ask:** Reliability engineering question — tests RTO/RPO understanding.

### ■ Key Concepts

- RTO (Recovery Time Objective): Maximum acceptable downtime after a disaster.
- RPO (Recovery Point Objective): Maximum acceptable data loss (measured in time).

### ■ Data Backup

- RDS: Automated daily snapshots + transaction log backups (point-in-time recovery to any second).
- S3: Versioning enabled; cross-region replication for critical buckets.
- EBS volumes: Automated snapshots via AWS Backup with retention policy.

### ■ Application / Infrastructure Backup

- Terraform state in S3 with versioning = infrastructure can be reproduced.
- Velero for Kubernetes cluster backup: backs up namespace resources and persistent volumes.
- Container images in ECR — immutable, always recoverable.

### ■ Disaster Recovery Strategies

- Backup & Restore: Cheapest; highest RTO. Restore from snapshots.
- Pilot Light: Minimal infra running in DR region; scale up on disaster.
- Warm Standby: Reduced-capacity replica always running; scale to full on failover.
- Multi-Site Active-Active: Full capacity in multiple regions; lowest RTO/RPO; highest cost.

---

★ *Tip: Always tie DR strategy to cost vs RTO/RPO trade-off — shows you think in business terms, not just tech.*

Question 19 of 20

## How do you prioritize tasks during a critical production outage?

**Why They Ask:** Behavioral + process question — tests incident management maturity.

### ■ Priority Order

- 1. RESTORE SERVICE first — rollback or redirect traffic. Don't debug with users impacted.
- 2. COMMUNICATE — notify stakeholders with a clear status update within 5 minutes.
- 3. INVESTIGATE root cause only after service is restored or stabilized.
- 4. DOCUMENT — write a post-mortem with timeline, root cause, and action items.

### ■ Incident Communication

- Use a dedicated incident Slack channel. Assign roles: Incident Commander, Comms Lead, Tech Lead.
- Update status page (if public-facing) every 15-30 minutes.
- Never go silent — even 'we're investigating' is better than no update.

### ■ What NOT to do

- Do not make random config changes hoping something works.
- Do not go dark while debugging.
- Do not skip the post-mortem — it's the most valuable part.

*★ Tip: Say 'restore first, understand second' — this is the industry standard and interviewers want to hear it explicitly.*

Question 20 of 20

## If you had to improve the existing DevOps process in an organization, what would you change first?

---

**Why They Ask:** Strategic thinking question — tests maturity and ability to assess systems holistically.

### ■ Step 1 – Observe Before Acting

- I would not assume what's broken. First, I'd audit: deployment frequency, lead time, MTTR, change failure rate (DORA metrics).
- Talk to developers: where is their biggest pain point?

### ■ Common High-Impact First Changes

- Automated testing in CI: If tests are missing or manual, every deployment is a risk. Add unit + integration tests first.
- Observability: If there's no monitoring/alerting, you're flying blind. Set up Prometheus + Grafana baseline first.
- Standardized environments: Dev/staging/prod parity via Docker + Helm prevents 'works on my machine' issues.
- IaC adoption: If infra is click-ops, migrate to Terraform to enable repeatability and audit trails.

### ■ My Honest First Move

- I'd fix observability first — you can't improve what you can't measure. Once you can see failures clearly, prioritizing the next fix becomes data-driven, not opinion-driven.

---

*★ Tip: Lead with 'I'd audit first' — jumping to solutions without diagnosis is a red flag. Interviewers want measured thinking.*

# You've Got This.

Every answer in this guide is grounded in your real internship work. Lead with honesty, structure your answers with STAR, and show ownership. Interviewers hire engineers they'd trust with production — be that person.

---

GitHub: github.com/Sanket006 | Target Roles: Junior DevOps · Cloud · SRE · Platform Engineer